Developing the Winning Model

In [1]:
```
import numpy as np
import pandas as pd
#%pip install yfinance
import yfinance as yf
import warnings
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
import zipfile
import requests
warnings.filterwarnings("ignore")
```

In [2]:
```
import os, subprocess

REPO_URL = "https://github.com/tongyuguo/HelpHerInvest.git"
REPO_DIR = "HelpHerInvest"
data_dir = os.path.join(REPO_DIR, "Data")

def clone_or_pull():
    if os.path.isdir(os.path.join(REPO_DIR, ".git")):
        subprocess.run(["git", "-C", REPO_DIR, "pull"])
    else:
        subprocess.run(["git", "clone", REPO_URL])

clone_or_pull()
```

```
Updating 29f2278..cbdd04e
Fast-forward
 Notebooks/Week 08/week08_third_model.ipynb         | 2147 ++++++++
 .../week07_random_forest_model01.ipynb             |  328 +-
 .../Tongyu Playground/llm.ipynb                    |  404 --
 .../Tongyu Playground/week08_third_model.ipynb     | 2147 ++++++++
 .../Tongyu Playground/working open ai llm.ipynb    |  290 ++
 .../week09_developing_winning_modell01.ipynb       | 5178 ++++++++++++
++++++++
 Reports/Week 07 /temp.html                         |    1 -
 .../week08_helpherinvest_written_report.pdf        |  Bin 0 -> 279062
bytes
 Reports/Week 08/week08_third_model.pdf             |  Bin 0 -> 60433 b
ytes
 9 files changed, 9926 insertions(+), 569 deletions(-)
 create mode 100644 Notebooks/Week 08/week08_third_model.ipynb
 delete mode 100644 Playground/Week 08 Develop 3rd modeling approach (e
ven more complex)/Tongyu Playground/llm.ipynb
 create mode 100644 Playground/Week 08 Develop 3rd modeling approach (e
ven more complex)/Tongyu Playground/week08_third_model.ipynb
 create mode 100644 Playground/Week 08 Develop 3rd modeling approach (e
ven more complex)/Tongyu Playground/working open ai llm.ipynb
 create mode 100644 Playground/Week 09 Select the winning model/Chris P
layground/week09_developing_winning_modell01.ipynb
 delete mode 100644 Reports/Week 07 /temp.html
 create mode 100644 Reports/Week 08/week08_helpherinvest_written_repor
t.pdf
 create mode 100644 Reports/Week 08/week08_third_model.pdf
```

```
From https://github.com/tongyuguo/HelpHerInvest
   29f2278..cbdd04e  main       -> origin/main
```

In [3]:

```python
import pandas as pd
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
#%pip install sentence-transformers
from sentence_transformers import SentenceTransformer
import zipfile

STOCKS_CSV = "HelpHerInvest/Data/nlp_clean_stock_symbols.csv"

def load_stocks(path=STOCKS_CSV):
    df = pd.read_csv(path)

    # Choose a text field for retrieval
    df["kw_text"] = df["document_clean_tfidf"].fillna(df["document_raw
    df["sem_text"] = df["document_raw"].fillna(df["document_clean_tfid

    # Keep only usable rows
    df = df[(df["kw_text"].str.len() > 0) & (df["sem_text"].str.len()
    return df

def build_hybrid_index(df,
```

```
                        sbert_model="sentence-transformers/all-MiniLM-L
                        tfidf_min_df=2,
                        tfidf_ngram_range=(1,2)):
    # TF-IDF for keyword similarity
    vectorizer = TfidfVectorizer(min_df=tfidf_min_df, max_df=0.95, ngr
    X_tfidf = vectorizer.fit_transform(df["kw_text"].astype(str))

    # SBERT embeddings for semantic similarity
    model = SentenceTransformer(sbert_model)
    X_emb = model.encode(
        df["sem_text"].astype(str).tolist(),
        batch_size=128,
        show_progress_bar=True,
        normalize_embeddings=True
    ).astype(np.float32)

    return vectorizer, X_tfidf, model, X_emb

df_stocks = load_stocks(STOCKS_CSV)
vectorizer, X_tfidf, sbert, X_emb = build_hybrid_index(df_stocks)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please
set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights:   0%|          | 0/103 [00:00<?, ?it/s]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/archit
ecture; not ok if you expect identical arch.
Batches:   0%|          | 0/81 [00:00<?, ?it/s]
```

In [4]:

```
import numpy as np

def _minmax(x):
    x = np.asarray(x, dtype=np.float32)
    mn, mx = x.min(), x.max()
    return (x - mn) / (mx - mn + 1e-9)

def build_query_from_df_text_inputs_row(row):
    # Use text_input if present; otherwise combine topics
    t = str(row.get("text_input", "")).strip()
    if t:
        return t
    p = str(row.get("primary_topic", "")).strip()
    s = str(row.get("secondary_topic", "")).strip()
    return (p + " " + s).strip()

def hybrid_retrieve(query, df_stocks, vectorizer, X_tfidf, sbert, X_em
                    top_k=50,
```

```
                    w_sem=0.65, w_kw=0.35,
                    mmr_lambda=0.80,
                    candidate_pool=500,
                    sector_cap=10):
    """
    w_sem/w_kw: weight semantic vs keyword similarity
    mmr_lambda: higher => more relevance, lower => more diversity
    candidate_pool: retrieve this many by relevance before diversifyin
    sector_cap: max results per sector (set None to disable)
    """

    # keyword similarity (TF-IDF cosine for L2-normalized TF-IDF is do
    q_tfidf = vectorizer.transform([query])
    kw_scores = (X_tfidf @ q_tfidf.T).toarray().ravel()

    # semantic similarity (cosine via normalized vectors)
    q_emb = sbert.encode([query], normalize_embeddings=True).astype(np
    sem_scores = (X_emb @ q_emb.T).ravel()

    # combine (normalize each so neither dominates)
    kw_n = _minmax(kw_scores)
    sem_n = _minmax(sem_scores)
    scores = w_sem * sem_n + w_kw * kw_n

    # candidate set
    pool = min(candidate_pool, len(df_stocks))
    cand_idx = np.argpartition(-scores, pool-1)[:pool]
    cand_idx = cand_idx[np.argsort(-scores[cand_idx])]

    # MMR diversification using embedding similarity to reduce redunda
    selected = []
    selected_set = set()

    # Precompute candidate embeddings
    cand_emb = X_emb[cand_idx]

    # Sector tracking
    sector_counts = {}
    def sector_ok(i):
        if sector_cap is None:
            return True
        sec = df_stocks.loc[i, "sector"]
        sec = sec if isinstance(sec, str) and sec else "Unknown"
        return sector_counts.get(sec, 0) < sector_cap

    for _ in range(min(top_k, len(cand_idx))):
        best_i = None
        best_mmr = -1e9

        for pos, i in enumerate(cand_idx):
            if i in selected_set:
                continue
```

```
            if not sector_ok(i):
                continue

            rel = scores[i]

            if not selected:
                mmr = rel
            else:
                # max similarity to already selected (redundancy)
                sim_to_selected = np.max(X_emb[i] @ X_emb[selected].T)
                mmr = mmr_lambda * rel - (1 - mmr_lambda) * sim_to_sel

            if mmr > best_mmr:
                best_mmr = mmr
                best_i = i

        if best_i is None:
            break

        selected.append(best_i)
        selected_set.add(best_i)
        sec = df_stocks.loc[best_i, "sector"]
        sec = sec if isinstance(sec, str) and sec else "Unknown"
        sector_counts[sec] = sector_counts.get(sec, 0) + 1

    out = df_stocks.loc[selected, ["symbol","company_name","sector","i
    out["hybrid_score"] = scores[selected]
    out = out.sort_values("hybrid_score", ascending=False).reset_index
    return out
```

In [5]:

```
import os
print(os.getcwd())
print(os.path.exists("HelpHerInvest/Data/final_dataset_20260224v2.csv.
print(os.path.exists("Data/final_dataset_20260224v2.csv.zip"))
```

```
/home/jupyter-guotq
True
False
```

In [6]:

```
# Load the data from the zip file

## PATH NEEDS TO BE UPDATED TO GITHUB DATA PATH
zip_path2 = "HelpHerInvest/Data/final_dataset_20260224v2.csv.zip"
with zipfile.ZipFile(zip_path2) as z:
    df = pd.read_csv(z.open("final_dataset_20260224v2.csv"))

## ## ## ##

tickers = df["Ticker"].unique()
print(f"Unique tickers in final dataset: {len(tickers)}")
print("Size of dataset:",
"Rows:",df.shape[0],
"Columns:",df.shape[1])
```

```
Unique tickers in final dataset: 1993
Size of dataset: Rows: 302024 Columns: 18
```

In [7]:
```
# count number of positive fwd_excess_return
print("Number of positive fwd_excess_return:", (df["fwd_excess"] > 0).
print("Number of negative fwd_excess_return:", (df["fwd_excess"] < 0).
print("Number of zero fwd_excess_return:", (df["fwd_excess"] == 0).sum

df = df.copy()

df['y'] = (df['fwd_excess'] > 0).astype(int)
```

```
Number of positive fwd_excess_return: 143948
Number of negative fwd_excess_return: 151730
Number of zero fwd_excess_return: 287
```

In [8]:
```
# Ensure datetime
df['Date'] = pd.to_datetime(df['Date'])

# Find latest month in dataset
max_date = df['Date'].max()

# Compute cutoff
cutoff_date = max_date - pd.DateOffset(months=2)

# Subset
df_last3 = df[df['Date'] > cutoff_date].copy()
df_last3.head()
```

Out[8]:

	Date	Ticker	mom_1m	mom_3m	mom_6m	mom_12m	mom_12m_
296052	2025-12-31	NVDA	0.4632	-0.1615	-0.0187	-0.0007	-(
296053	2025-12-31	GOOGL	-0.2588	0.9727	0.2777	0.0626	
296054	2025-12-31	AAPL	-0.2902	0.1099	0.0543	-0.0705	-(
296055	2025-12-31	MSFT	-0.2136	-0.4135	-0.1202	-0.0553	-(
296056	2025-12-31	AMZN	-0.1489	0.0413	-0.0823	-0.0795	-(

Split the Data 70/15/15

In [9]:
```
def time_split(df, date_col='Date', train_frac=0.7, val_frac=0.15):
    df = df.sort_values(date_col).copy()
    unique_dates = df[date_col].drop_duplicates().sort_values().to_num
```

```
    # retreive the num of rows to create train/val/test split
    n = len(unique_dates)
    train_end = int(n * train_frac)
    val_end = int(n * (train_frac + val_frac))

    train_dates = unique_dates[:train_end]
    val_dates   = unique_dates[train_end:val_end]
    test_dates  = unique_dates[val_end:]

    train = df[df[date_col].isin(train_dates)]
    val   = df[df[date_col].isin(val_dates)]
    test  = df[df[date_col].isin(test_dates)]

    return train, val, test

training_data, validation_data, testing_data = time_split(df)
print("Train size:", training_data.shape)
print("Val size:", validation_data.shape)
print("Test size:", testing_data.shape)
training_data.tail()
```

```
Train size: (193638, 19)
Val size: (52704, 19)
Test size: (55682, 19)
```

Out[9]:

	Date	Ticker	mom_1m	mom_3m	mom_6m	mom_12m	mom_12m_ex
192479	2021-04-30	SSNC	0.2723	0.1806	-0.0387	-0.1580	-0.
192485	2021-04-30	LH	0.0402	0.0731	-0.0345	-0.0866	-0.(
192486	2021-04-30	IHG	-0.0824	0.0408	-0.0311	-0.1020	-0.(
192487	2021-04-30	CHD	-0.6758	-0.6346	-0.0551	-0.1904	-0.1
192484	2021-04-30	NVR	0.3069	-0.0899	-0.0381	-0.0860	-0.(

Preprocess the Data

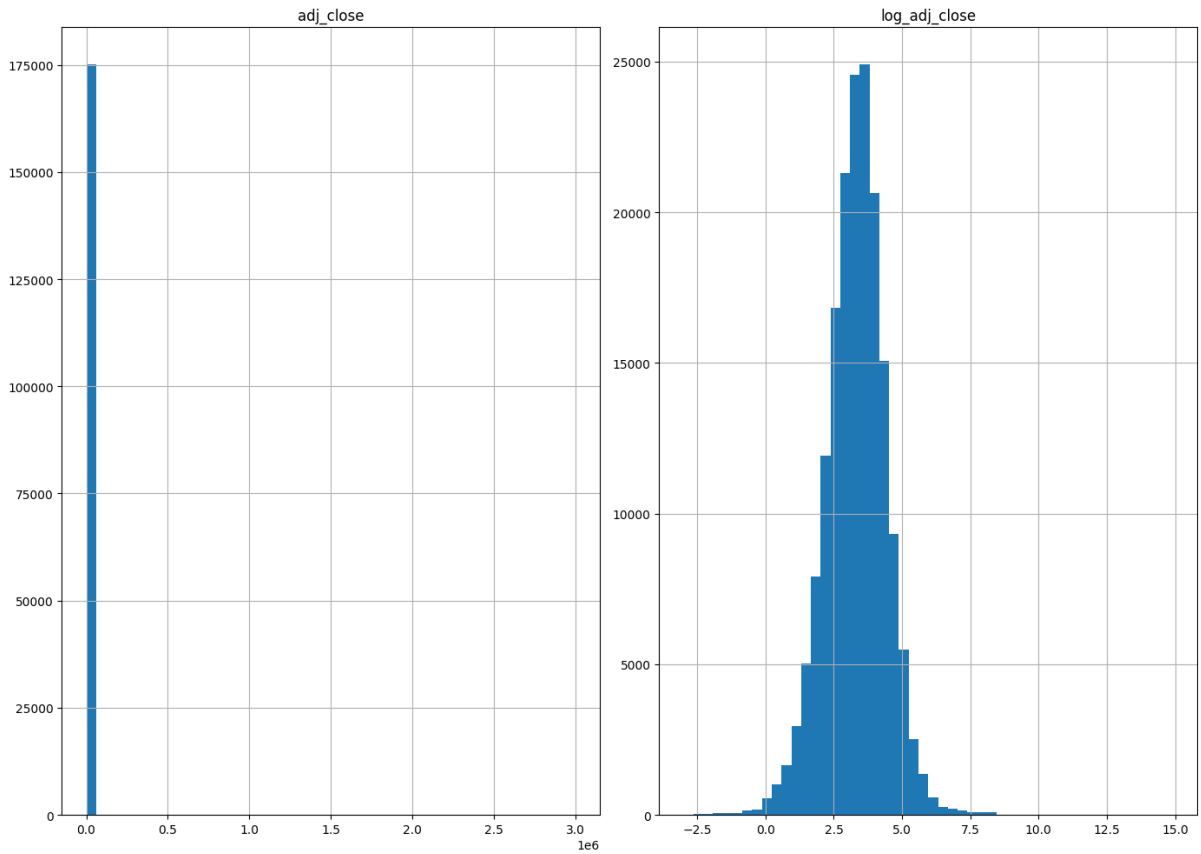
In [10]:

```
def preprocess_data(df):

    # preprocess the data by dropping rows with missing values
    print("Missing fwd_excess values:")
```

```
    df_dropped = df.dropna()
    print("---  ---  ---")
    print("Length of original:",len(df.index))
    print("Length of dropped:",len(df_dropped.index))
    print("Sanity check - missing fwd_excess values after dropping:")
    print(df_dropped[["Date","Ticker","fwd_excess"]].tail(5))



    # Create log_adj_close column
    df_dropped["log_adj_close"] = np.log(df_dropped["adj_close"])
    df_dropped[["adj_close","log_adj_close"]].hist(figsize=(14, 10), b
    plt.tight_layout()
    plt.show()

    return df_dropped
```

In [11]:
```
train_df = preprocess_data(training_data)
train_df.head(5)
```

```
Missing fwd_excess values:
---  ---  ---
Length of original: 193638
Length of dropped: 175057
Sanity check - missing fwd_excess values after dropping:
             Date Ticker  fwd_excess
192479 2021-04-30   SSNC      0.0042
192485 2021-04-30     LH      0.0596
192486 2021-04-30    IHG     -0.1253
192487 2021-04-30    CHD     -0.0415
192484 2021-04-30    NVR     -0.0135
```



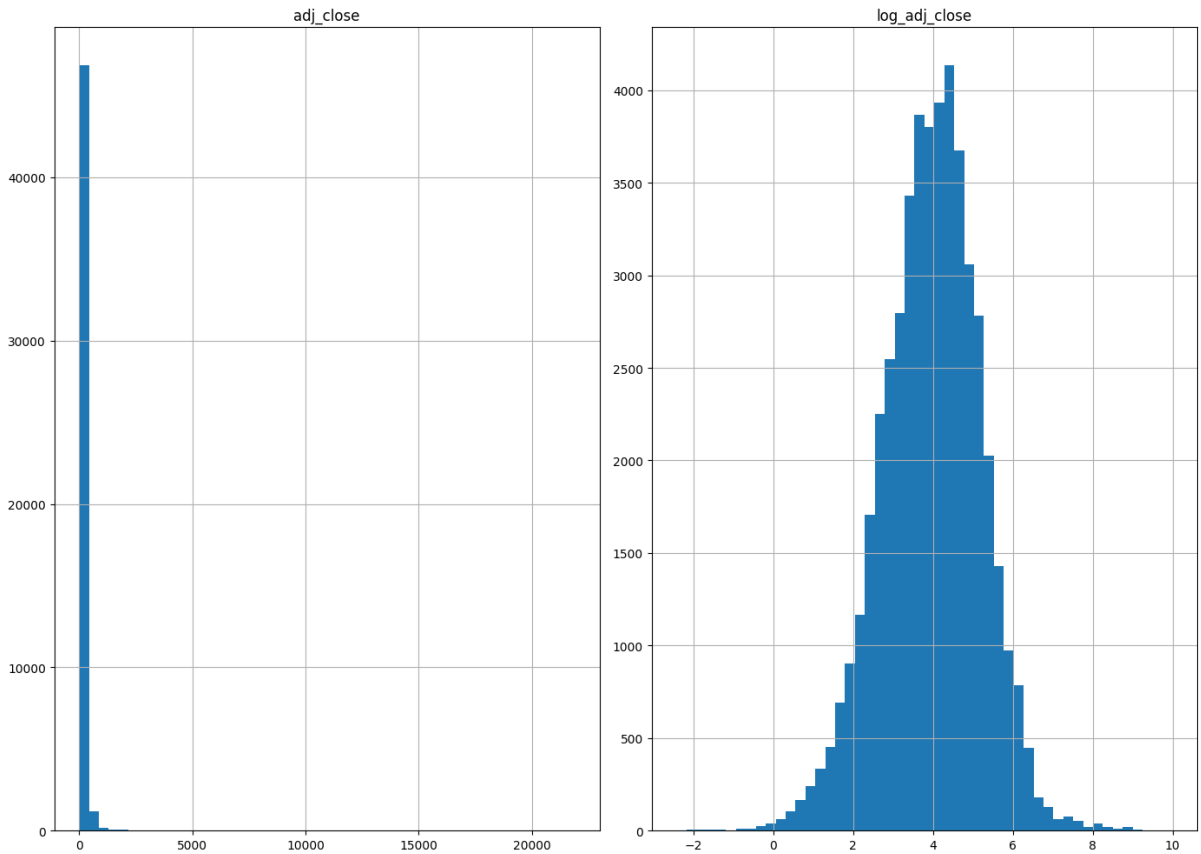
Out[11]:

	Date	Ticker	mom_1m	mom_3m	mom_6m	mom_12m	mom_12m_ex_1
14213	2011-01-31	TTMI	0.4972	2.4448	1.1222	0.4989	0.32
14216	2011-01-31	DSGX	-0.2733	-0.0527	-0.1622	-0.4035	-0.34
14215	2011-01-31	CHDN	-0.6391	0.2684	-0.2432	-0.4573	-0.28
14214	2011-01-31	GSAT	-0.9448	-1.6521	-1.5770	0.1845	0.55
14218	2011-01-31	MASI	0.1469	-0.5690	0.3651	-0.3740	-0.44

In [12]:

```
test_df = preprocess_data(testing_data)
test_df.head(5)
```

```
Missing fwd_excess values:
---  ---  ---
Length of original: 55682
Length of dropped: 48493
Sanity check - missing fwd_excess values after dropping:
            Date Ticker  fwd_excess
294730 2025-11-30    GPN      0.0285
294731 2025-11-30   BURL      0.2126
294732 2025-11-30   KTOS      0.1828
294733 2025-11-30   TRMB     -0.1891
294728 2025-11-30   SBAC      0.0290
```



Out[12]:

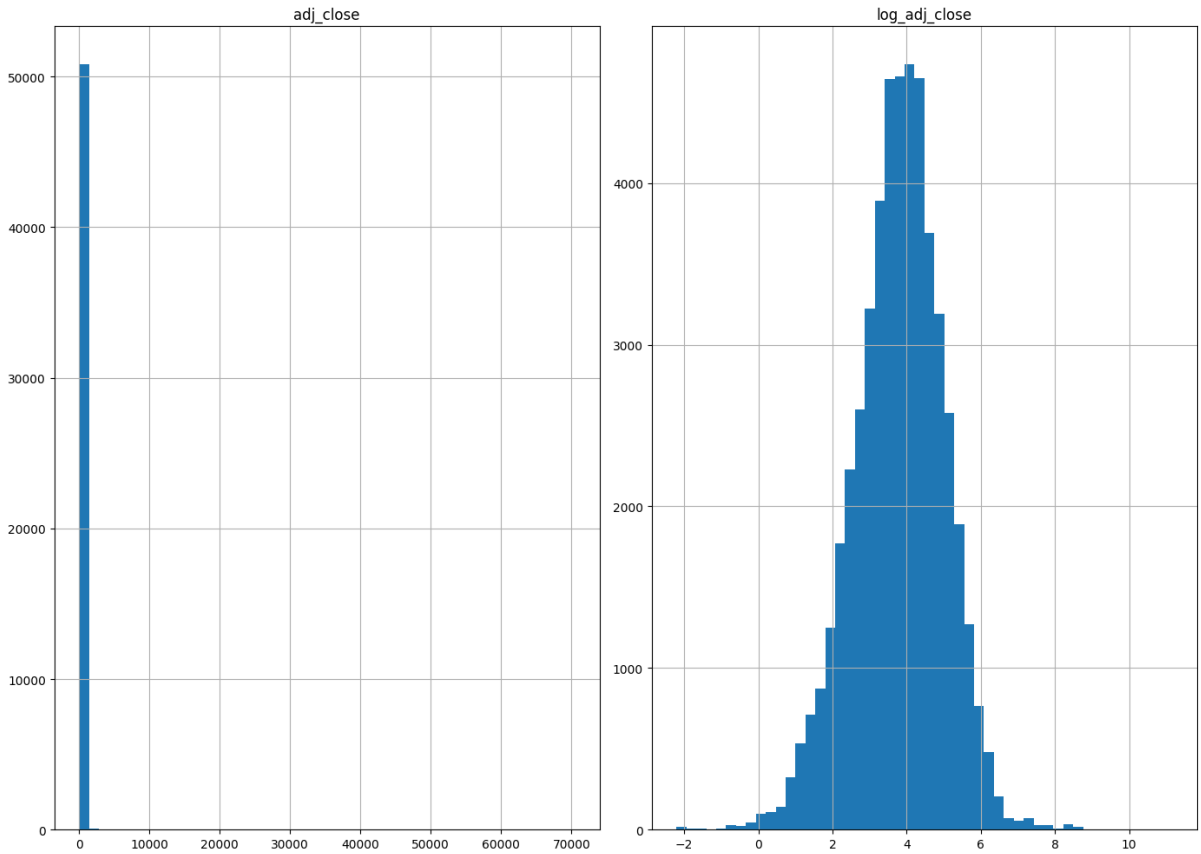
	Date	Ticker	mom_1m	mom_3m	mom_6m	mom_12m	mom_12m_e
247583	2023-10-31	MNSO	0.2820	1.8365	1.5085	10.5307	10.
247584	2023-10-31	ACT	0.6388	0.7340	0.6076	0.1936	0.
247585	2023-10-31	ZETA	-0.1585	-0.1970	-0.7219	-0.3790	-0.
247586	2023-10-31	RHP	0.7997	0.1445	-0.0652	-0.2143	-0.
247592	2023-10-31	ARW	-0.4508	-0.4682	-0.0130	0.1119	0.

In [13]:

```
val_df = preprocess_data(validation_data)
```

```python
val_df.head(5)
```

```
Missing fwd_excess values:
---  ---  ---
Length of original: 52704
Length of dropped: 50961
Sanity check - missing fwd_excess values after dropping:
            Date Ticker  fwd_excess
245100 2023-09-30    HPQ      0.0652
245099 2023-09-30    PFG     -0.0149
245098 2023-09-30   TWLO      0.1799
245097 2023-09-30   CHRW     -0.1058
245108 2023-09-30    RCI      0.1129
```



Out[13]:

	Date	Ticker	mom_1m	mom_3m	mom_6m	mom_12m	mom_12m_ex
194796	2021-05-31	SITE	-0.6464	-0.0770	-0.0860	-0.0469	-0.
194804	2021-05-31	DLB	-0.6009	-0.5411	-0.3627	-0.0460	-0.
194803	2021-05-31	M	0.8177	0.5672	1.0082	0.2009	0.
194805	2021-05-31	TRNO	-0.3711	0.2303	-0.3603	-0.1159	-0.
194802	2021-05-31	DAR	-0.3743	-0.0735	0.2600	0.2133	0.

Create the Train, Validation, and Test sets

In [14]:

```
feature_cols = [col for col in train_df.columns if col not in ["Date",

X_train, y_train = train_df[feature_cols], train_df['y']
X_val, y_val     = val_df[feature_cols],   val_df['y']
X_test, y_test   = test_df[feature_cols],  test_df['y']
```

Logistic Regression Models

In [15]:

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import numpy as np

# L1 (Lasso) Logistic Regression

log_reg = Pipeline([
    ('scaler', StandardScaler()),
    ('logit', LogisticRegression(
        penalty='l1',
        solver='saga',
        max_iter=1000
    ))
])

log_reg.fit(X_train, y_train)
```

Out[15]:

```
▸ Pipeline
  ▸ StandardScaler
  ▸ LogisticRegression
```

In [16]:

```python
# Elastic Net Logistic Regression

elastic_net = Pipeline([
    ('scaler', StandardScaler()),
    ('logit', LogisticRegression(
        penalty='elasticnet',
        solver='saga',
        l1_ratio=0.5,   # 50% L1 / 50% L2
        max_iter=1000
    ))
])

elastic_net.fit(X_train, y_train)
```

Out[16]:

```
▸ Pipeline
  ▸ StandardScaler
  ▸ LogisticRegression
```

In [17]:

```python
# Weighted Logistic Regression

weight_log_reg = Pipeline([
    ('scaler', StandardScaler()),
    ('logit', LogisticRegression(
        class_weight='balanced',
        max_iter=500
    ))
])

weight_log_reg.fit(X_train, y_train)
```

Out[17]:

```
▸ Pipeline
  ▸ StandardScaler
  ▸ LogisticRegression
```

Light Gradient Boosting Machine (LightGBM) Models

In [18]:
```
# LightGBM Model 1 with more conservative hyperparameters

import lightgbm as lgb

lgb_model1 = lgb.LGBMClassifier(
    n_estimators=1000,
    learning_rate=0.01,
    max_depth=6,
    num_leaves=31
)

lgb_model1.fit(
    X_train,
    y_train,
    eval_set=[(X_val, y_val)],

)
```

```
[LightGBM] [Info] Number of positive: 86255, number of negative: 88802
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead
of testing was 0.015706 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 3825
[LightGBM] [Info] Number of data points in the train set: 175057, numbe
r of used features: 15
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.492725 -> initscore=-
0.029101
[LightGBM] [Info] Start training from score -0.029101
```

Out[18]:

▾ LGBMClassifier ⓘ

▸ Parameters

In [19]:
```
# LightGBM Model 2 with more aggressive hyperparameters

lgb_model2 = lgb.LGBMClassifier(
    n_estimators=1000,
    learning_rate=0.01,
    max_depth=12,
    num_leaves=31,
    class_weight="balanced",
)

lgb_model2.fit(
    X_train,
    y_train,
    eval_set=[(X_val, y_val)],

)
```

```
[LightGBM] [Info] Number of positive: 86255, number of negative: 88802
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead
of testing was 0.013883 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 3825
[LightGBM] [Info] Number of data points in the train set: 175057, numbe
r of used features: 15
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=-
0.000000
[LightGBM] [Info] Start training from score -0.000000
```

Out[19]:

```
▾ LGBMClassifier ⓘ
▸ Parameters
```

In [20]:

```python
# LightGBM Model 3 with even more aggressive hyperparameters

lgb_model3 = lgb.LGBMClassifier(
    n_estimators=500,
    learning_rate=0.01,
    max_depth=6,
    num_leaves=20
)

lgb_model3.fit(
    X_train,
    y_train,
    eval_set=[(X_val, y_val)],

)
```

```
[LightGBM] [Info] Number of positive: 86255, number of negative: 88802
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead
of testing was 0.013777 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 3825
[LightGBM] [Info] Number of data points in the train set: 175057, numbe
r of used features: 15
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.492725 -> initscore=-
0.029101
[LightGBM] [Info] Start training from score -0.029101
```

Out[20]:

```
▾ LGBMClassifier ⓘ
▸ Parameters
```

Neural Network Models

In [21]:

```python
import tensorflow as tf
from tensorflow import keras
```

```
from tensorflow.keras import layers, callbacks, regularizers
from sklearn.utils.class_weight import compute_class_weight

SEED = 42
tf.random.set_seed(SEED)
```

```
2026-03-26 01:54:35.243116: I tensorflow/core/platform/cpu_feature_guar
d.cc:210] This TensorFlow binary is optimized to use available CPU inst
ructions in performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, re
build TensorFlow with the appropriate compiler flags.
```

In [22]:

```
'''
X_train, y_train = train_df[feature_cols], train_df['y']
X_val, y_val     = val_df[feature_cols],   val_df['y']
X_test, y_test   = test_df[feature_cols],  test_df['y']
'''
X_train_nn = X_train.values
y_train_nn = y_train.values
X_val_nn = X_val.values
y_val_nn = y_val.values
X_test_nn = X_test.values
y_test_nn = y_test.values

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train_nn)
X_val_s = scaler.transform(X_val_nn)
X_test_s = scaler.transform(X_test_nn)
```

In [23]:

```
# Class weights for imbalance
classes = np.unique(y_train_nn)
weights = compute_class_weight(class_weight="balanced", classes=classe
class_weight = dict(zip(classes, weights))
class_weight
```

Out[23]:

```
{np.int64(0): np.float64(0.9856591067768744),
 np.int64(1): np.float64(1.0147643614862907)}
```

In [24]:

```
early_stop = callbacks.EarlyStopping(
    monitor="val_loss",
    patience=20,
    restore_best_weights=True
)

reduce_lr = callbacks.ReduceLROnPlateau(
    monitor="val_loss",
    factor=0.5,
    patience=4,
    min_lr=1e-5
)
```

In [25]:

```
from sklearn.metrics import (
```

```
    accuracy_score,
    f1_score,
    classification_report,
    roc_auc_score,
    confusion_matrix
)

def evaluate_model(model, X, y, name="Model"):
    prob = model.predict(X, verbose=0).ravel()
    pred = (prob >= 0.5).astype(int)

    print(f"\n{name}")
    print("Accuracy:", round(accuracy_score(y, pred), 4))
    print("F1:", round(f1_score(y_test, pred, average='macro'), 4))
    print("ROC AUC:", round(roc_auc_score(y, prob), 4))
    print("\nConfusion Matrix")
    print(confusion_matrix(y, pred))
    print("\nClassification Report")
    print(classification_report(y, pred, digits=4))
```

In [26]:

```
def build_mlp_baseline(input_dim):
    model = keras.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(64, activation="relu"),
        layers.Dropout(0.2),
        layers.Dense(32, activation="relu"),
        layers.Dropout(0.2),
        layers.Dense(1, activation="sigmoid")
    ])

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=1e-3),
        loss="binary_crossentropy",
        metrics=[
            keras.metrics.BinaryAccuracy(name="accuracy"),
            keras.metrics.AUC(name="auc")
        ]
    )
    return model

model_base = build_mlp_baseline(X_train_s.shape[1])

history_base = model_base.fit(
    X_train_s, y_train,
    validation_data=(X_val_s, y_val),
    epochs=100,
    batch_size=256,
    class_weight=class_weight,
    callbacks=[early_stop, reduce_lr],
    verbose=1
)
```

```
2026-03-26 01:54:48.914323: E external/local_xla/xla/stream_executor/cu
da/cuda_platform.cc:51] failed call to cuInit: INTERNAL: CUDA error: Fa
iled call to cuInit: UNKNOWN ERROR (303)
Epoch 1/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.5072 - auc: 0.51
08 - loss: 0.6953 - val_accuracy: 0.5048 - val_auc: 0.5051 - val_loss:
0.6936 - learning_rate: 0.0010
Epoch 2/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.5127 - auc: 0.52
02 - loss: 0.6923 - val_accuracy: 0.4952 - val_auc: 0.5000 - val_loss:
0.6945 - learning_rate: 0.0010
Epoch 3/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.5150 - auc: 0.52
47 - loss: 0.6918 - val_accuracy: 0.4941 - val_auc: 0.4978 - val_loss:
0.6948 - learning_rate: 0.0010
Epoch 4/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.5168 - auc: 0.52
68 - loss: 0.6916 - val_accuracy: 0.4951 - val_auc: 0.5010 - val_loss:
0.6952 - learning_rate: 0.0010
Epoch 5/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.5188 - auc: 0.53
11 - loss: 0.6912 - val_accuracy: 0.4952 - val_auc: 0.4994 - val_loss:
0.6951 - learning_rate: 0.0010
Epoch 6/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.5209 - auc: 0.53
41 - loss: 0.6908 - val_accuracy: 0.4973 - val_auc: 0.5000 - val_loss:
0.6951 - learning_rate: 5.0000e-04
Epoch 7/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.5213 - auc: 0.53
60 - loss: 0.6905 - val_accuracy: 0.4966 - val_auc: 0.4989 - val_loss:
0.6955 - learning_rate: 5.0000e-04
Epoch 8/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.5226 - auc: 0.53
71 - loss: 0.6903 - val_accuracy: 0.4958 - val_auc: 0.4978 - val_loss:
0.6956 - learning_rate: 5.0000e-04
Epoch 9/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.5250 - auc: 0.53
76 - loss: 0.6903 - val_accuracy: 0.4983 - val_auc: 0.4983 - val_loss:
0.6956 - learning_rate: 5.0000e-04
Epoch 10/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.5238 - auc: 0.53
79 - loss: 0.6901 - val_accuracy: 0.4980 - val_auc: 0.4992 - val_loss:
0.6955 - learning_rate: 2.5000e-04
Epoch 11/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.5239 - auc: 0.53
92 - loss: 0.6900 - val_accuracy: 0.4989 - val_auc: 0.4994 - val_loss:
0.6955 - learning_rate: 2.5000e-04
Epoch 12/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.5259 - auc: 0.53
99 - loss: 0.6899 - val_accuracy: 0.5003 - val_auc: 0.5008 - val_loss:
0.6954 - learning_rate: 2.5000e-04
Epoch 13/100
```

```
684/684 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.5252 - auc: 0.54
04 - loss: 0.6898 - val_accuracy: 0.5001 - val_auc: 0.5006 - val_loss:
0.6954 - learning_rate: 2.5000e-04
Epoch 14/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.5239 - auc: 0.54
04 - loss: 0.6897 - val_accuracy: 0.5029 - val_auc: 0.5010 - val_loss:
0.6955 - learning_rate: 1.2500e-04
Epoch 15/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.5253 - auc: 0.54
08 - loss: 0.6898 - val_accuracy: 0.5030 - val_auc: 0.5018 - val_loss:
0.6954 - learning_rate: 1.2500e-04
Epoch 16/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.5251 - auc: 0.54
11 - loss: 0.6897 - val_accuracy: 0.5007 - val_auc: 0.5011 - val_loss:
0.6956 - learning_rate: 1.2500e-04
Epoch 17/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.5261 - auc: 0.54
19 - loss: 0.6897 - val_accuracy: 0.5004 - val_auc: 0.5006 - val_loss:
0.6957 - learning_rate: 1.2500e-04
Epoch 18/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.5290 - auc: 0.54
46 - loss: 0.6893 - val_accuracy: 0.5008 - val_auc: 0.5007 - val_loss:
0.6957 - learning_rate: 6.2500e-05
Epoch 19/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.5257 - auc: 0.54
21 - loss: 0.6896 - val_accuracy: 0.5016 - val_auc: 0.5009 - val_loss:
0.6955 - learning_rate: 6.2500e-05
Epoch 20/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.5259 - auc: 0.54
21 - loss: 0.6895 - val_accuracy: 0.5017 - val_auc: 0.5010 - val_loss:
0.6955 - learning_rate: 6.2500e-05
Epoch 21/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.5273 - auc: 0.54
32 - loss: 0.6895 - val_accuracy: 0.5020 - val_auc: 0.5013 - val_loss:
0.6955 - learning_rate: 6.2500e-05
```

In [27]:

```
def build_mlp_regularized(input_dim):
    l2 = regularizers.l2(1e-4)

    inputs = keras.Input(shape=(input_dim,))
    x = layers.Dense(256, kernel_regularizer=l2)(inputs)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.Dropout(0.35)(x)

    x = layers.Dense(128, kernel_regularizer=l2)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.Dropout(0.30)(x)

    x = layers.Dense(64, kernel_regularizer=l2)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
```

```
    x = layers.Dropout(0.20)(x)

    outputs = layers.Dense(1, activation="sigmoid")(x)

    model = keras.Model(inputs, outputs)
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=5e-4),
        loss="binary_crossentropy",
        metrics=[
            keras.metrics.BinaryAccuracy(name="accuracy"),
            keras.metrics.AUC(name="auc")
        ]
    )
    return model

model_reg = build_mlp_regularized(X_train_s.shape[1])

history_reg = model_reg.fit(
    X_train_s, y_train,
    validation_data=(X_val_s, y_val),
    epochs=100,
    batch_size=256,
    class_weight=class_weight,
    callbacks=[early_stop, reduce_lr],
    verbose=1
)
```

```
Epoch 1/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 10s 10ms/step - accuracy: 0.5037 - auc: 0.
5051 - loss: 0.7403 - val_accuracy: 0.4916 - val_auc: 0.4945 - val_los
s: 0.7223 - learning_rate: 5.0000e-04
Epoch 2/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.5079 - auc: 0.5
104 - loss: 0.7217 - val_accuracy: 0.4965 - val_auc: 0.4978 - val_loss:
0.7182 - learning_rate: 5.0000e-04
Epoch 3/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.5115 - auc: 0.5
166 - loss: 0.7155 - val_accuracy: 0.4953 - val_auc: 0.4994 - val_loss:
0.7145 - learning_rate: 5.0000e-04
Epoch 4/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.5142 - auc: 0.5
204 - loss: 0.7108 - val_accuracy: 0.5008 - val_auc: 0.5010 - val_loss:
0.7103 - learning_rate: 5.0000e-04
Epoch 5/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.5186 - auc: 0.5
262 - loss: 0.7073 - val_accuracy: 0.4983 - val_auc: 0.5030 - val_loss:
0.7084 - learning_rate: 2.5000e-04
Epoch 6/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 6s 9ms/step - accuracy: 0.5184 - auc: 0.52
75 - loss: 0.7052 - val_accuracy: 0.4997 - val_auc: 0.5042 - val_loss:
0.7065 - learning_rate: 2.5000e-04
Epoch 7/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 6s 9ms/step - accuracy: 0.5202 - auc: 0.52
```

```
96 - loss: 0.7031 - val_accuracy: 0.4977 - val_auc: 0.5026 - val_loss:
0.7049 - learning_rate: 2.5000e-04
Epoch 8/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.5219 - auc: 0.5
340 - loss: 0.7010 - val_accuracy: 0.4999 - val_auc: 0.5019 - val_loss:
0.7038 - learning_rate: 2.5000e-04
Epoch 9/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.5242 - auc: 0.5
346 - loss: 0.6998 - val_accuracy: 0.4970 - val_auc: 0.5017 - val_loss:
0.7032 - learning_rate: 1.2500e-04
Epoch 10/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.5226 - auc: 0.5
356 - loss: 0.6989 - val_accuracy: 0.4993 - val_auc: 0.5033 - val_loss:
0.7025 - learning_rate: 1.2500e-04
Epoch 11/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.5259 - auc: 0.5
383 - loss: 0.6981 - val_accuracy: 0.4993 - val_auc: 0.5034 - val_loss:
0.7021 - learning_rate: 1.2500e-04
Epoch 12/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.5258 - auc: 0.5
391 - loss: 0.6975 - val_accuracy: 0.5007 - val_auc: 0.5051 - val_loss:
0.7016 - learning_rate: 1.2500e-04
Epoch 13/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.5282 - auc: 0.5
421 - loss: 0.6967 - val_accuracy: 0.5004 - val_auc: 0.5046 - val_loss:
0.7013 - learning_rate: 6.2500e-05
Epoch 14/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.5278 - auc: 0.5
415 - loss: 0.6965 - val_accuracy: 0.5014 - val_auc: 0.5046 - val_loss:
0.7011 - learning_rate: 6.2500e-05
Epoch 15/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.5289 - auc: 0.5
433 - loss: 0.6961 - val_accuracy: 0.5015 - val_auc: 0.5036 - val_loss:
0.7012 - learning_rate: 6.2500e-05
Epoch 16/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.5287 - auc: 0.5
426 - loss: 0.6961 - val_accuracy: 0.5022 - val_auc: 0.5048 - val_loss:
0.7008 - learning_rate: 6.2500e-05
Epoch 17/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 9ms/step - accuracy: 0.5286 - auc: 0.54
33 - loss: 0.6957 - val_accuracy: 0.5038 - val_auc: 0.5047 - val_loss:
0.7008 - learning_rate: 3.1250e-05
Epoch 18/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.5296 - auc: 0.5
443 - loss: 0.6955 - val_accuracy: 0.5039 - val_auc: 0.5050 - val_loss:
0.7007 - learning_rate: 3.1250e-05
Epoch 19/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.5289 - auc: 0.5
436 - loss: 0.6955 - val_accuracy: 0.5034 - val_auc: 0.5046 - val_loss:
0.7007 - learning_rate: 3.1250e-05
Epoch 20/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 9ms/step - accuracy: 0.5300 - auc: 0.54
```

```
56 - loss: 0.6950 - val_accuracy: 0.5048 - val_auc: 0.5050 - val_loss:
0.7006 - learning_rate: 3.1250e-05
```

In [28]:

```python
def build_mlp_regularized_deep(input_dim):
    l2 = regularizers.l2(1e-4)

    inputs = keras.Input(shape=(input_dim,))
    x = layers.Dense(256, kernel_regularizer=l2)(inputs)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.Dropout(0.35)(x)

    x = layers.Dense(128, kernel_regularizer=l2)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.Dropout(0.30)(x)

    x = layers.Dense(64, kernel_regularizer=l2)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.Dropout(0.10)(x)

    x = layers.Dense(32, kernel_regularizer=l2)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.Dropout(0.05)(x)

    x = layers.Dense(16, kernel_regularizer=l2)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)

    outputs = layers.Dense(1, activation="sigmoid")(x)

    model = keras.Model(inputs, outputs)
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=5e-4),
        loss="binary_crossentropy",
        metrics=[
            keras.metrics.BinaryAccuracy(name="accuracy"),
            keras.metrics.AUC(name="auc")
        ]
    )
    return model

model_reg_deep = build_mlp_regularized_deep(X_train_s.shape[1])

history_reg_deep = model_reg_deep.fit(
    X_train_s, y_train,
    validation_data=(X_val_s, y_val),
    epochs=100,
    batch_size=256,
    class_weight=class_weight,
    callbacks=[early_stop, reduce_lr],
```

```
    verbose=1
)
```

```
Epoch 1/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 14s 13ms/step - accuracy: 0.5013 - auc: 0.
5030 - loss: 0.7337 - val_accuracy: 0.4968 - val_auc: 0.4918 - val_los
s: 0.7262 - learning_rate: 5.0000e-04
Epoch 2/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 11ms/step - accuracy: 0.5064 - auc: 0.5
101 - loss: 0.7237 - val_accuracy: 0.4921 - val_auc: 0.4941 - val_loss:
0.7222 - learning_rate: 5.0000e-04
Epoch 3/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 11ms/step - accuracy: 0.5126 - auc: 0.5
186 - loss: 0.7183 - val_accuracy: 0.4823 - val_auc: 0.4911 - val_loss:
0.7183 - learning_rate: 5.0000e-04
Epoch 4/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 11ms/step - accuracy: 0.5146 - auc: 0.5
220 - loss: 0.7132 - val_accuracy: 0.4834 - val_auc: 0.4908 - val_loss:
0.7135 - learning_rate: 5.0000e-04
Epoch 5/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 11ms/step - accuracy: 0.5171 - auc: 0.5
263 - loss: 0.7093 - val_accuracy: 0.4912 - val_auc: 0.4973 - val_loss:
0.7109 - learning_rate: 2.5000e-04
Epoch 6/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 8s 11ms/step - accuracy: 0.5194 - auc: 0.5
292 - loss: 0.7068 - val_accuracy: 0.4906 - val_auc: 0.4980 - val_loss:
0.7092 - learning_rate: 2.5000e-04
Epoch 7/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 8s 11ms/step - accuracy: 0.5212 - auc: 0.5
321 - loss: 0.7045 - val_accuracy: 0.4917 - val_auc: 0.4937 - val_loss:
0.7075 - learning_rate: 2.5000e-04
Epoch 8/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 9s 13ms/step - accuracy: 0.5218 - auc: 0.5
328 - loss: 0.7028 - val_accuracy: 0.4970 - val_auc: 0.4943 - val_loss:
0.7060 - learning_rate: 2.5000e-04
Epoch 9/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 17s 25ms/step - accuracy: 0.5244 - auc: 0.
5359 - loss: 0.7013 - val_accuracy: 0.4977 - val_auc: 0.4970 - val_los
s: 0.7053 - learning_rate: 1.2500e-04
Epoch 10/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 17s 25ms/step - accuracy: 0.5257 - auc: 0.
5389 - loss: 0.7003 - val_accuracy: 0.4976 - val_auc: 0.4979 - val_los
s: 0.7052 - learning_rate: 1.2500e-04
Epoch 11/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 8s 11ms/step - accuracy: 0.5275 - auc: 0.5
403 - loss: 0.6994 - val_accuracy: 0.4988 - val_auc: 0.4966 - val_loss:
0.7044 - learning_rate: 1.2500e-04
Epoch 12/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 11ms/step - accuracy: 0.5287 - auc: 0.5
417 - loss: 0.6990 - val_accuracy: 0.4987 - val_auc: 0.4961 - val_loss:
0.7040 - learning_rate: 1.2500e-04
Epoch 13/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 8s 11ms/step - accuracy: 0.5300 - auc: 0.5
```

```
437 - loss: 0.6983 - val_accuracy: 0.4996 - val_auc: 0.4977 - val_loss:
0.7043 - learning_rate: 6.2500e-05
Epoch 14/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 11ms/step - accuracy: 0.5300 - auc: 0.5
435 - loss: 0.6981 - val_accuracy: 0.4978 - val_auc: 0.4965 - val_loss:
0.7041 - learning_rate: 6.2500e-05
Epoch 15/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 11ms/step - accuracy: 0.5293 - auc: 0.5
427 - loss: 0.6979 - val_accuracy: 0.4999 - val_auc: 0.4958 - val_loss:
0.7038 - learning_rate: 6.2500e-05
Epoch 16/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 11ms/step - accuracy: 0.5293 - auc: 0.5
432 - loss: 0.6978 - val_accuracy: 0.4988 - val_auc: 0.4966 - val_loss:
0.7038 - learning_rate: 6.2500e-05
Epoch 17/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 11ms/step - accuracy: 0.5316 - auc: 0.5
460 - loss: 0.6972 - val_accuracy: 0.5001 - val_auc: 0.4970 - val_loss:
0.7034 - learning_rate: 3.1250e-05
Epoch 18/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 11ms/step - accuracy: 0.5310 - auc: 0.5
455 - loss: 0.6971 - val_accuracy: 0.4998 - val_auc: 0.4968 - val_loss:
0.7034 - learning_rate: 3.1250e-05
Epoch 19/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 11ms/step - accuracy: 0.5304 - auc: 0.5
459 - loss: 0.6969 - val_accuracy: 0.5006 - val_auc: 0.4974 - val_loss:
0.7034 - learning_rate: 3.1250e-05
Epoch 20/100
684/684 ━━━━━━━━━━━━━━━━━━━━ 7s 11ms/step - accuracy: 0.5312 - auc: 0.5
459 - loss: 0.6969 - val_accuracy: 0.5009 - val_auc: 0.4974 - val_loss:
0.7034 - learning_rate: 3.1250e-05
```

In [29]:

```python
from sklearn.metrics import f1_score, classification_report, confusion

## Used to Evaluate NN models ##
def evaluate_nn_model(model, X, y, name="Model"):
    prob = model.predict(X, verbose=0).ravel()
    pred = (prob >= 0.5).astype(int)

    print(f"\n{name}")
    print("Accuracy:", round(accuracy_score(y, pred), 4))
    print("F1:", round(f1_score(y, pred, average='macro'), 4))
    print("ROC AUC:", round(roc_auc_score(y, prob), 4))
    print("\nConfusion Matrix")
    print(confusion_matrix(y, pred))
    print("\nClassification Report")
    print(classification_report(y, pred, digits=4))


## Used to Evaluate the Logistic Regression and LightGBM models ##
def evaluate(model, X_test, y_test):
    y_pred = model.predict(X_test)
    probs = model.predict_proba(X_test)[:,1]
```

```
    print("Classification Report:\n")
    print(classification_report(y_test, y_pred))
    print("F1:", round(f1_score(y_test, y_pred, average='macro'), 4))
    print("ROC-AUC:", round(roc_auc_score(y_test, probs, average='macr


print("Logistic Regression - Standard:")
evaluate(log_reg, X_val, y_val)
print("\n-----  -----  -----  -----  -----  -----  -----  -----  -----
print("Elastic Net Logistic Regression:")
evaluate(elastic_net, X_val, y_val)
print("\n-----  -----  -----  -----  -----  -----  -----  -----  -----
print("Weighted Logistic Regression:")
evaluate(weight_log_reg, X_val, y_val)
print("LightGBM Model 1:")
evaluate(lgb_model1, X_val, y_val)
print("\n-----  -----  -----  -----  -----  -----  -----  -----  -----
print("LightGBM Model 2:")
evaluate(lgb_model2, X_val, y_val)
print("\n-----  -----  -----  -----  -----  -----  -----  -----  -----
print("LightGBM Model 3:")
evaluate(lgb_model3, X_val, y_val)
print("\n-----  -----  -----  -----  -----  -----  -----  -----  -----
print("Neural Network Baseline:")
evaluate_nn_model(model_base, X_val_s, y_val_nn, name="NN Baseline")
print("\n-----  -----  -----  -----  -----  -----  -----  -----  -----
print("Neural Network Regularized:")
evaluate_nn_model(model_reg,X_val_s, y_val_nn, name="NN Regularized")
print("\n-----  -----  -----  -----  -----  -----  -----  -----  -----
print("Neural Network Regularized Deep:")
evaluate_nn_model(model_reg_deep, X_val_s, y_val_nn, name="NN Regulari
```

```
Logistic Regression - Standard:
Classification Report:

              precision    recall  f1-score   support

           0       0.53      0.85      0.66     26843
           1       0.50      0.16      0.24     24118

    accuracy                           0.53     50961
   macro avg       0.52      0.51      0.45     50961
weighted avg       0.52      0.53      0.46     50961

F1: 0.45
ROC-AUC: 0.5143

-----  -----  -----  -----  -----  -----  -----  -----  -----  -----
Elastic Net Logistic Regression:
Classification Report:

              precision    recall  f1-score   support
```

```
           0       0.53      0.85      0.66     26843
           1       0.50      0.16      0.24     24118

    accuracy                           0.53     50961
   macro avg       0.52      0.51      0.45     50961
weighted avg       0.52      0.53      0.46     50961

F1: 0.4501
ROC-AUC: 0.5143

-----  -----  -----  -----  -----  -----  -----  -----  -----  -----
Weighted Logistic Regression:
Classification Report:

              precision    recall  f1-score   support

           0       0.53      0.66      0.59     26843
           1       0.49      0.35      0.41     24118

    accuracy                           0.52     50961
   macro avg       0.51      0.51      0.50     50961
weighted avg       0.51      0.52      0.51     50961

F1: 0.5008
ROC-AUC: 0.5143
LightGBM Model 1:
Classification Report:

              precision    recall  f1-score   support

           0       0.53      0.62      0.57     26843
           1       0.49      0.40      0.44     24118

    accuracy                           0.52     50961
   macro avg       0.51      0.51      0.51     50961
weighted avg       0.51      0.52      0.51     50961

F1: 0.5063
ROC-AUC: 0.5129

-----  -----  -----  -----  -----  -----  -----  -----  -----  -----
LightGBM Model 2:
Classification Report:

              precision    recall  f1-score   support

           0       0.54      0.53      0.53     26843
           1       0.48      0.49      0.48     24118

    accuracy                           0.51     50961
   macro avg       0.51      0.51      0.51     50961
weighted avg       0.51      0.51      0.51     50961
```

```
F1: 0.5088
ROC-AUC: 0.5134

-----  -----  -----  -----  -----  -----  -----  -----  -----  -----
LightGBM Model 3:
Classification Report:

              precision    recall  f1-score   support

           0       0.53      0.67      0.59     26843
           1       0.48      0.35      0.41     24118

    accuracy                           0.52     50961
   macro avg       0.51      0.51      0.50     50961
weighted avg       0.51      0.52      0.50     50961

F1: 0.4991
ROC-AUC: 0.5144

-----  -----  -----  -----  -----  -----  -----  -----  -----  -----
Neural Network Baseline:

NN Baseline
Accuracy: 0.5048
F1: 0.5047
ROC AUC: 0.5054

Confusion Matrix
[[13207 13636]
 [11598 12520]]

Classification Report
              precision    recall  f1-score   support

           0     0.5324    0.4920    0.5114     26843
           1     0.4787    0.5191    0.4981     24118

    accuracy                         0.5048     50961
   macro avg     0.5055    0.5056    0.5047     50961
weighted avg     0.5070    0.5048    0.5051     50961


-----  -----  -----  -----  -----  -----  -----  -----  -----  -----
Neural Network Regularized:

NN Regularized
Accuracy: 0.4916
F1: 0.4866
ROC AUC: 0.4943

Confusion Matrix
```

```
[[10018 16825]
 [ 9085 15033]]

Classification Report
              precision    recall  f1-score   support

           0     0.5244    0.3732    0.4361     26843
           1     0.4719    0.6233    0.5371     24118

    accuracy                         0.4916     50961
   macro avg     0.4981    0.4983    0.4866     50961
weighted avg     0.4996    0.4916    0.4839     50961


-----  -----  -----  -----  -----  -----  -----  -----  -----  -----
Neural Network Regularized Deep:

NN Regularized Deep
Accuracy: 0.4968
F1: 0.4947
ROC AUC: 0.4915

Confusion Matrix
[[14291 12552]
 [13091 11027]]

Classification Report
              precision    recall  f1-score   support

           0     0.5219    0.5324    0.5271     26843
           1     0.4677    0.4572    0.4624     24118

    accuracy                         0.4968     50961
   macro avg     0.4948    0.4948    0.4947     50961
weighted avg     0.4962    0.4968    0.4965     50961
```

In [30]:

```
'''
def evaluate_to_df(model, X_test, y_test, model_name):
    y_pred = model.predict(X_test)
    probs = model.predict_proba(X_test)[:, 1]

    # Classification report as table
    report_dict = classification_report(y_test, y_pred, output_dict=Tr
    report_df = pd.DataFrame(report_dict).T

    # Add summary metrics
    report_df.loc["overall", "f1_marco"] = f1_score(y_test, y_pred, av
    report_df.loc["overall", "roc_auc"] = roc_auc_score(y_test, probs,
    report_df["model"] = model_name

    return report_df
```

```
results = {
    "Logistic Regression": evaluate_to_df(log_reg, X_val, y_val, "Logi
    "Elastic Net Logistic Regression": evaluate_to_df(elastic_net, X_v
    "Weighted Logistic Regression": evaluate_to_df(weight_log_reg, X_v
    "LightGBM Model 1": evaluate_to_df(lgb_model1, X_val, y_val, "Ligh
    "LightGBM Model 2": evaluate_to_df(lgb_model2, X_val, y_val, "Ligh
    "LightGBM Model 3": evaluate_to_df(lgb_model3, X_val, y_val, "Ligh
}

## THIS NEEDS TO BE UPDATED TO GITHUB OUTPUT PATH
class_report_output = r"/Users/christoomey/Desktop/Courses/ADAN8888_Ap

# one sheet per model
with pd.ExcelWriter(class_report_output, engine="openpyxl") as writer:
    for sheet_name, df in results.items():
        df.to_excel(writer, sheet_name=sheet_name)

print(f"Saved to {class_report_output}")
'''
```

Out[30]:

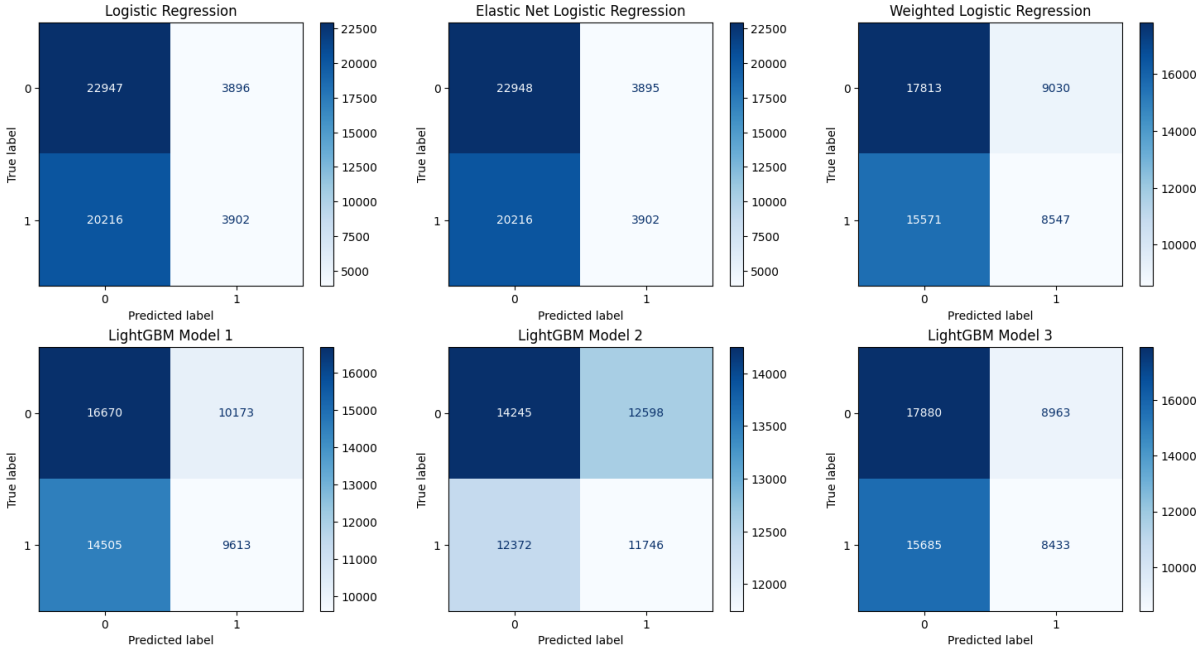
```
'\ndef evaluate_to_df(model, X_test, y_test, model_name):\n    y_pred
= model.predict(X_test)\n    probs = model.predict_proba(X_test)[:,
1]\n\n    # Classification report as table\n    report_dict = classif
ication_report(y_test, y_pred, output_dict=True)\n    report_df = pd.
DataFrame(report_dict).T\n\n    # Add summary metrics\n    report_df.
loc["overall", "f1_marco"] = f1_score(y_test, y_pred, average=\'macro
\')\n    report_df.loc["overall", "roc_auc"] = roc_auc_score(y_test,
probs, average=\'macro\')\n    report_df["model"] = model_name\n\n
return report_df\n\nresults = {\n    "Logistic Regression": evaluate_
to_df(log_reg, X_val, y_val, "Logistic Regression"),\n    "Elastic Ne
t Logistic Regression": evaluate_to_df(elastic_net, X_val, y_val, "El
astic Net Logistic Regression"),\n    "Weighted Logistic Regression":
evaluate_to_df(weight_log_reg, X_val, y_val, "Weighted Logistic Regre
ssion"),\n    "LightGBM Model 1": evaluate_to_df(lgb_model1, X_val, y
_val, "LightGBM Model 1"),\n    "LightGBM Model 2": evaluate_to_df(lg
b_model2, X_val, y_val, "LightGBM Model 2"),\n    "LightGBM Model 3":
evaluate_to_df(lgb_model3, X_val, y_val, "LightGBM Model 3"),\n}\n\
n## THIS NEEDS TO BE UPDATED TO GITHUB OUTPUT PATH\nclass_report_outp
ut = r"/Users/christoomey/Desktop/Courses/ADAN8888_Applied_Analytics_
Project/Output/classification_reports_week09.xlsx"\n\n# one sheet per
model\nwith pd.ExcelWriter(class_report_output, engine="openpyxl") as
writer:\n    for sheet_name, df in results.items():\n        df.to_ex
cel(writer, sheet_name=sheet_name)\n\nprint(f"Saved to {class_report_
output}")\n'
```

In [31]:

```
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

models = {
    "Logistic Regression": log_reg,
    "Elastic Net Logistic Regression": elastic_net,
    "Weighted Logistic Regression": weight_log_reg,
    "LightGBM Model 1": lgb_model1,
```

```python
    "LightGBM Model 2": lgb_model2,
    "LightGBM Model 3": lgb_model3
}

# Create confusion matrix tables
cm_tables = {}
for name, model in models.items():

    y_pred = model.predict(X_val)
    cm = confusion_matrix(y_val, y_pred)  # [[TN, FP], [FN, TP]]
    cm_df = pd.DataFrame(cm,
                         index=["Actual 0", "Actual 1"],
                         columns=["Pred 0", "Pred 1"])
    cm_tables[name] = cm_df
    print(f"\n{name}\n{cm_df}")

# Plot confusion matrix for each model
fig, axes = plt.subplots(2, 3, figsize=(15, 8))

for ax, (name, model) in zip(axes.flat, models.items()):
    y_pred = model.predict(X_val)
    ConfusionMatrixDisplay.from_predictions(
        y_val, y_pred, ax=ax, cmap="Blues", values_format="d"
    )
    ax.set_title(name)

plt.tight_layout()
plt.show()
```

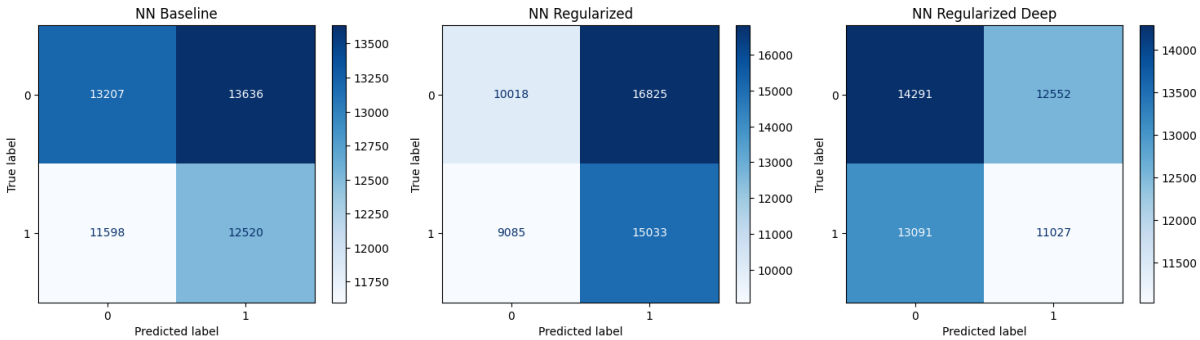
```
Logistic Regression
          Pred 0  Pred 1
Actual 0   22947    3896
Actual 1   20216    3902

Elastic Net Logistic Regression
          Pred 0  Pred 1
Actual 0   22948    3895
Actual 1   20216    3902

Weighted Logistic Regression
          Pred 0  Pred 1
Actual 0   17813    9030
Actual 1   15571    8547

LightGBM Model 1
          Pred 0  Pred 1
Actual 0   16670   10173
Actual 1   14505    9613

LightGBM Model 2
          Pred 0  Pred 1
Actual 0   14245   12598
Actual 1   12372   11746

LightGBM Model 3
          Pred 0  Pred 1
Actual 0   17880    8963
Actual 1   15685    8433
```



In [32]:
```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```python
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

nn_models = {
    "NN Baseline": model_base,
    "NN Regularized": model_reg,
    "NN Regularized Deep": model_reg_deep
}

X_nn = X_val_s

# make sure y is 1D
y_true = np.asarray(y_val).ravel()

# Create confusion matrix tables
cm_tables_nn = {}

for name, model in nn_models.items():
    prob = model.predict(X_nn, verbose=0).ravel()
    pred = (prob >= 0.5).astype(int)

    cm = confusion_matrix(y_true, pred)
    cm_df = pd.DataFrame(
        cm,
        index=["Actual 0", "Actual 1"],
        columns=["Pred 0", "Pred 1"]
    )

    cm_tables_nn[name] = cm_df
    print(f"\n{name}\n{cm_df}")

# Plot confusion matrices
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for ax, (name, model) in zip(axes.flat, nn_models.items()):
    prob = model.predict(X_nn, verbose=0).ravel()
    pred = (prob >= 0.5).astype(int)

    ConfusionMatrixDisplay.from_predictions(
        y_true,
        pred,
        ax=ax,
        cmap="Blues",
        values_format="d"
    )
    ax.set_title(name)

plt.tight_layout()
plt.show()
```

```
NN Baseline
          Pred 0  Pred 1
Actual 0   13207   13636
Actual 1   11598   12520

NN Regularized
          Pred 0  Pred 1
Actual 0   10018   16825
Actual 1    9085   15033

NN Regularized Deep
          Pred 0  Pred 1
Actual 0   14291   12552
Actual 1   13091   11027
```



In [33]:
```python
from sklearn.metrics import log_loss

def get_model_errors(model, X_train, y_train, X_val, y_val):
    train_probs = model.predict_proba(X_train)[:, 1]
    val_probs = model.predict_proba(X_val)[:, 1]

    train_loss = log_loss(y_train, train_probs)
    val_loss = log_loss(y_val, val_probs)

    return train_loss, val_loss
```

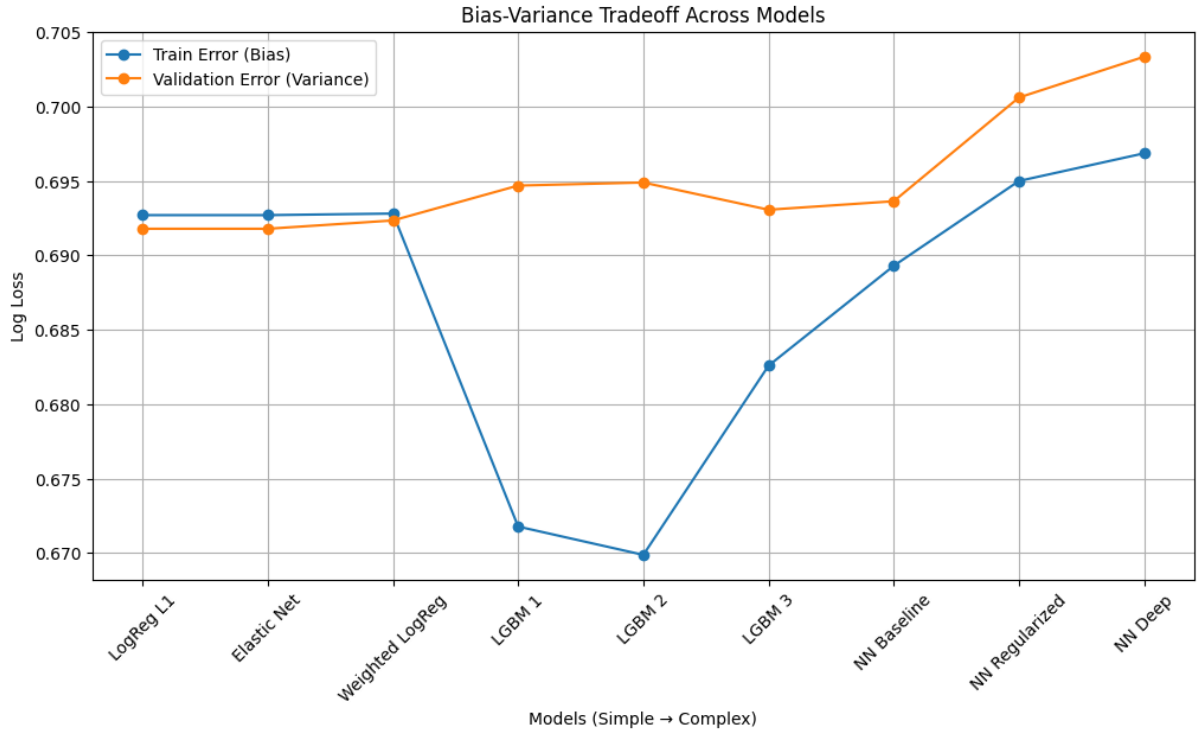
In [34]:
```python
def get_nn_errors(history):
    train_loss = min(history.history['loss'])
    val_loss = min(history.history['val_loss'])
    return train_loss, val_loss
```

In [35]:
```python
results = []

# Logistic + LGBM
models = [
    ("LogReg L1", log_reg),
    ("Elastic Net", elastic_net),
    ("Weighted LogReg", weight_log_reg),
    ("LGBM 1", lgb_model1),
    ("LGBM 2", lgb_model2),
    ("LGBM 3", lgb_model3),
]
```

```
for name, model in models:
    train_loss, val_loss = get_model_errors(model, X_train, y_train, X
    results.append((name, train_loss, val_loss))

# Neural Networks
nn_models = [
    ("NN Baseline", history_base),
    ("NN Regularized", history_reg),
    ("NN Deep", history_reg_deep),
]

for name, history in nn_models:
    train_loss, val_loss = get_nn_errors(history)
    results.append((name, train_loss, val_loss))
```

In [36]:

```
import matplotlib.pyplot as plt

model_names = [r[0] for r in results]
train_errors = [r[1] for r in results]
val_errors = [r[2] for r in results]

plt.figure(figsize=(12,6))

plt.plot(model_names, train_errors, marker='o', label='Train Error (Bi
plt.plot(model_names, val_errors, marker='o', label='Validation Error

plt.xticks(rotation=45)
plt.ylabel("Log Loss")
plt.xlabel("Models (Simple → Complex)")
plt.title("Bias-Variance Tradeoff Across Models")
plt.legend()
plt.grid(True)

plt.show()
```



In [37]:
```
'''
print("\n-----  -----  -----  -----  -----  -----  -----  -----  -----
print("NN Baseline - Test Set:")
evaluate_nn_model(model_base, X_test_s, y_test_nn, name="NN Baseline")

print("\n-----  -----  -----  -----  -----  -----  -----  -----  -----
print("NN Regularized - Test Set:")
evaluate_nn_model(model_reg, X_test_s, y_test_nn, name="NN Regularized

print("\n-----  -----  -----  -----  -----  -----  -----  -----  -----
print("NN Regularized Deep- Test Set:")
evaluate_nn_model(model_reg_deep, X_test_s, y_test_nn, name="NN Regula
'''
```

Out[37]:
```
'\nprint("\n-----  -----  -----  -----  -----  -----  -----  -----  -
----  -----")\nprint("NN Baseline - Test Set:")\nevaluate_nn_model(mo
del_base, X_test_s, y_test_nn, name="NN Baseline")\n\nprint("\n-----
-----  -----  -----  -----  -----  -----  -----  -----")\nprin
t("NN Regularized - Test Set:")\nevaluate_nn_model(model_reg, X_test_
s, y_test_nn, name="NN Regularized")\n\nprint("\n-----  -----  -----
-----  -----  -----  -----  -----  -----  -----")\nprint("NN Regulari
zed Deep- Test Set:")\nevaluate_nn_model(model_reg_deep, X_test_s, y_
test_nn, name="NN Regularized Deep")\n'
```

In [38]:
```
# Selected the best model from validation performance and evaluate on
print("\n-----  -----  -----  -----  -----  -----  -----  -----  -----
print("LightGBM Model 2 - Test Set:")
evaluate(lgb_model2, X_test, y_test)
```

```
-----  -----  -----  -----  -----  -----  -----  -----  -----  -----
LightGBM Model 2 - Test Set:
Classification Report:

              precision    recall  f1-score   support

           0       0.55      0.53      0.54     25892
           1       0.48      0.50      0.49     22601

    accuracy                           0.52     48493
   macro avg       0.52      0.52      0.52     48493
weighted avg       0.52      0.52      0.52     48493

F1: 0.5155
ROC-AUC: 0.5215
```